

ValueListenableBuilder – Templates Grid Documentation

Overview

This widget section uses `ValueListenableBuilder<TextEditingValue>` to rebuild a templates grid automatically whenever the search input changes. It is commonly used with a `TextEditingController` to reactively update UI without using `setState()`.

In this case, it listens to `_searchController` and updates a **GridView of templates** based on filtered search results.

Why ValueListenableBuilder?

`ValueListenableBuilder` is useful when: - You want **lightweight reactive UI updates** - You want to avoid unnecessary `setState()` calls - You are listening to `TextEditingController`, `ValueNotifier`, etc.

It rebuilds **only the widget subtree**, not the whole screen.

Widget Structure Breakdown

1. ValueListenableBuilder

```
ValueListenableBuilder<TextEditingValue>(  
  valueListenable: _searchController,  
  builder: (context, value, _) {  
    final results = filteredTemplates;  
  })
```

Explanation

- **valueListenable** → `_searchController`
- **value** → Current `TextEditingValue` (search text)
- **filteredTemplates** → List updated based on search query

Every time the search text changes, the builder runs again.

2. Empty State UI

```
if (results.isEmpty) {  
  return Center(  
    child: Padding(  
      padding: EdgeInsets.only(top: 40.h),  
      child: Text(  
        'No template found',  
        style: TextStyle.textStyle14w400c637381sora,  
      ),  
    ),  
  );  
}
```

Purpose

- Shows a **friendly empty state** when no templates match the search
- Prevents rendering an empty grid

3. GridView Builder

```
return GridView.builder(  
  shrinkWrap: true,  
  physics: const NeverScrollableScrollPhysics(),  
  itemCount: results.length,  
)
```

Key Properties

Property	Purpose
<code>shrinkWrap: true</code>	Allows GridView inside another scroll view
<code>NeverScrollableScrollPhysics</code>	Prevents nested scrolling issues
<code>itemCount</code>	Total filtered templates

4. Grid Layout Configuration

```
SliverGridDelegateWithFixedCrossAxisCount(  
  crossAxisCount: 2,  
  crossAxisSpacing: 10,  
  mainAxisSpacing: 10,  
)
```

```
        childAspectRatio: 1 / 1.684,  
    )
```

Layout Behavior

- **2 columns** grid
- Consistent spacing between items
- Fixed aspect ratio for uniform cards

5. Grid Item Widget

```
final item = results[index];  
  
return TemplatesWidgets(  
    imagePath: item.thumbnailUrl ??  
        'https://images.pexels.com/photos/358255/pexels-photo-358255.jpeg',  
    title: item.name ?? '',  
    category: item.category ?? '',  
    isPremium: !(item.isDefault ?? false),  
    onTap: () {},  
);
```

Data Mapping

UI Field	Data Source
Image	<code>thumbnailUrl</code> or fallback image
Title	<code>item.name</code>
Category	<code>item.category</code>
Premium Flag	<code>!item.isDefault</code>

Ensures **null safety** using fallback values.

How Search Filtering Works (Example)

```
List<Template> get filteredTemplates {  
    final query = _searchController.text.toLowerCase();  
  
    return allTemplates.where((template) {  
        return template.name
```

```
        ?.toLowerCase()
        .contains(query) ?? false;
    }).toList();
}
```

Advantages of This Approach

- ✓ No `setState()` needed
- ✓ Efficient rebuilds
- ✓ Clean separation of UI & logic
- ✓ Works great with search, filters, toggles

Common Mistakes to Avoid

- ✗ Using `GridView.builder` without `shrinkWrap` inside scroll views
- ✗ Forgetting `NeverScrollableScrollPhysics`
- ✗ Heavy logic inside the builder

Best Use Cases

- Search results
- Filterable grids or lists
- Live text input UI
- Settings toggles

Summary

This implementation: - Listens to text changes using `ValueListenableBuilder` - Dynamically updates a templates grid - Handles empty states gracefully - Keeps UI performant and clean

Perfect for **search-driven Flutter UIs** 🚫

If you want: - ☒ **Advanced optimization** -  **Animation on grid items** - **Debounced search**

Just tell me ♀